```
#Part 1: Market Setup
import random
students = [f"i{k}" for k in range(1, 19)]
schools  = [f"s{k}" for k in range(1, 4)]
capacity = {s: 6 for s in schools}
random.seed(123)
prefs = {}
for i in students:
    ranking = schools[:]
    random.shuffle(ranking)
    prefs[i] = ranking
priority = {}
for s in schools:
    ordering = students[:]
    random.shuffle(ordering)
    priority[s] = ordering
print("Capacities:")
for s in schools:
    print(f"  {s}: {capacity[s]}")
print("\nStudent preferences (succ_i):")
for i in students:
    print(f"  {i}: {prefs[i]}")
print("\nSchool priorities (succ_s):")
for s in schools:
    print(f"  {s}: {priority[s]}")
#Part 2: Matching Mechanisms
import random
import pandas as pd
students = [f"i{k}" for k in range(1, 19)]
schools  = [f"s{k}" for k in range(1, 4)]
capacity = {s: 6 for s in schools}
random.seed(123)
prefs = {}
for i in students:
    ranking = schools[:]
    random.shuffle(ranking)
    prefs[i] = ranking
priority = {}
for s in schools:
    ordering = students[:]
    random.shuffle(ordering)
    priority[s] = ordering
# (i) Deferred Acceptance (student-proposing)
def deferred_acceptance(students, schools, capacity, prefs, priority):
    rank = {s: {stu: idx for idx, stu in enumerate(priority[s])} for s in schools}
    next_choice = {i: 0 for i in students}
    held = {s: [] for s in schools}
    free = set(students)
```

```
    while free:
        i = free.pop()
        if next_choice[i] >= len(prefs[i]):
            continue
        s = prefs[i][next_choice[i]]
        next_choice[i] += 1
        held[s].append(i)
        if len(held[s]) > capacity[s]:
            held[s].sort(key=lambda stu: rank[s][stu])
            rejected = held[s][capacity[s]:]
            held[s] = held[s][:capacity[s]]
            free.update(rejected)
    matching = {i: None for i in students}
    for s, lst in held.items():
        for i in lst:
            matching[i] = s
    return matching
# (ii) Immediate Acceptance
def immediate_acceptance(students, schools, capacity, prefs, priority):
    rank = {s: {stu: idx for idx, stu in enumerate(priority[s])} for s in schools}
    remaining_cap = capacity.copy()
    assigned = {}
    unassigned = set(students)
    for r in range(len(schools)):
        apps = {s: [] for s in schools}
        for i in list(unassigned):
            s = prefs[i][r]
            apps[s].append(i)
        for s in schools:
            if remaining_cap[s] <= 0:
                continue
            if apps[s]:
                apps[s].sort(key=lambda stu: rank[s][stu])
                accept = apps[s][:remaining_cap[s]]
                for i in accept:
                    assigned[i] = s
                unassigned.difference_update(accept)
                remaining_cap[s] -= len(accept)
    return {i: assigned.get(i, None) for i in students}
#  (iii) TTC with priorities + capacities
def ttc_with_priorities(students, schools, capacity, prefs, priority):
    remaining_cap = capacity.copy()
    remaining_students = set(students)
    remaining_schools = {s for s in schools if remaining_cap[s] > 0}
    matching = {i: None for i in students}
    def top_choice_school(i):
        for s in prefs[i]:
            if s in remaining_schools:
```

```
            return s
    return None
def top_priority_student(s):
    for stu in priority[s]:
        if stu in remaining_students:
            return stu
    return None
while remaining_students and remaining_schools:
    # pointers
    stu_points = {i: top_choice_school(i) for i in remaining_students}
    sch_points = {s: top_priority_student(s) for s in remaining_schools}
    # find cycles in directed graph
    visited = set()
    cycles = []
    nodes = list(remaining_students) + list(remaining_schools)
    for start in nodes:
        if start in visited:
            continue
        path = []
        index = {}
        node = start
        while True:
            if node is None:
                for n in path:
                    visited.add(n)
                break
            if node in index:
                cycles.append(path[index[node]:])
                for n in path:
                    visited.add(n)
                break
            if node in visited:
                break
            index[node] = len(path)
            path.append(node)
            if node in remaining_students:
                node = stu_points[node]
            else:
                node = sch_points[node]
    assigned_this_round = set()
    for cyc in cycles:
        for node in cyc:
            if node in remaining_students:
                i = node
                s = stu_points[i]
                if s in remaining_schools:
                    matching[i] = s
                    assigned_this_round.add(i)
```

```
                    remaining_cap[s] -= 1
        remaining_students.difference_update(assigned_this_round)
        remaining_schools = {s for s in remaining_schools if remaining_cap[s] > 0}
    return matching
da  = deferred_acceptance(students, schools, capacity, prefs, priority)
ia  = immediate_acceptance(students, schools, capacity, prefs, priority)
ttc = ttc_with_priorities(students, schools, capacity, prefs, priority)
results = pd.DataFrame({
    "student": students,
    "DA":  [da[i] for i in students],
    "IA":  [ia[i] for i in students],
    "TTC": [ttc[i] for i in students],
})
print(results.to_string(index=False))
print("\nDeferred Acceptance (DA):")
for student in sorted(da.keys()):
    print(student, "→", da[student])
print("\nImmediate Acceptance (IA):")
for student in sorted(ia.keys()):
    print(student, "→", ia[student])
print("\nTop Trading Cycles (TTC):")
for student in sorted(ttc.keys()):
    print(student, "→", ttc[student])
results = pd.DataFrame({
    "Student": sorted(da.keys()),
    "DA":  [da[s] for s in sorted(da.keys())],
    "IA":  [ia[s] for s in sorted(da.keys())],
    "TTC": [ttc[s] for s in sorted(da.keys())],
})
print("\nFinal Matching Comparison:")
print(results)
results.to_csv("matching_results.csv", index=False)
print("DA assigned:", sum(v is not None for v in da.values()))
print("IA assigned:", sum(v is not None for v in ia.values()))
print("TTC assigned:", sum(v is not None for v in ttc.values()))
#Part 3: Efiiciency Analysis
import random
import pandas as pd
# Mechanisms
def deferred_acceptance(students, schools, capacity, prefs, priority):
    rank = {s: {stu: idx for idx, stu in enumerate(priority[s])} for s in schools}
    next_choice = {i: 0 for i in students}
    held = {s: [] for s in schools}
    free = set(students)
    while free:
        i = free.pop()
        if next_choice[i] >= len(prefs[i]):
            continue
```

```
            s = prefs[i][next_choice[i]]
            next_choice[i] += 1
            held[s].append(i)
            if len(held[s]) > capacity[s]:
                held[s].sort(key=lambda stu: rank[s][stu])
                rejected = held[s][capacity[s]:]
                held[s] = held[s][:capacity[s]]
                free.update(rejected)
    matching = {i: None for i in students}
    for s, lst in held.items():
        for i in lst:
            matching[i] = s
    return matching
def immediate_acceptance(students, schools, capacity, prefs, priority):
    rank = {s: {stu: idx for idx, stu in enumerate(priority[s])} for s in schools}
    remaining_cap = capacity.copy()
    assigned = {}
    unassigned = set(students)
    for r in range(len(schools)):
        apps = {s: [] for s in schools}
        for i in list(unassigned):
            s = prefs[i][r]
            apps[s].append(i)
        for s in schools:
            if remaining_cap[s] <= 0:
                continue
            if apps[s]:
                apps[s].sort(key=lambda stu: rank[s][stu])
                accept = apps[s][:remaining_cap[s]]
                for i in accept:
                    assigned[i] = s
                unassigned.difference_update(accept)
                remaining_cap[s] -= len(accept)
    return {i: assigned.get(i, None) for i in students}
def ttc_with_priorities(students, schools, capacity, prefs, priority):
    remaining_cap = capacity.copy()
    remaining_students = set(students)
    remaining_schools = {s for s in schools if remaining_cap[s] > 0}
    matching = {i: None for i in students}
    def top_choice_school(i):
        for s in prefs[i]:
            if s in remaining_schools:
                return s
        return None
    def top_priority_student(s):
        for stu in priority[s]:
            if stu in remaining_students:
                return stu
```

```
            return None
        while remaining_students and remaining_schools:
            stu_points = {i: top_choice_school(i) for i in remaining_students}
            sch_points = {s: top_priority_student(s) for s in remaining_schools}
            def nxt(node):
                if node in remaining_students:
                    return stu_points[node]
                else:
                    return sch_points[node]
            visited_global = set()
            assigned_this_round = set()
            for start in list(remaining_students) + list(remaining_schools):
                if start in visited_global:
                    continue
                path = []
                pos = {}
                node = start
                while node is not None and node not in visited_global and node not in pos:
                    pos[node] = len(path)
                    path.append(node)
                    node = nxt(node)
                if node is not None and node in pos:
                    cycle = path[pos[node]:]
                    for x in cycle:
                        if x in remaining_students:
                            i = x
                            s = stu_points[i]
                            matching[i] = s
                            assigned_this_round.add(i)
                            remaining_cap[s] -= 1
                for x in path:
                    visited_global.add(x)
            remaining_students.difference_update(assigned_this_round)
            remaining_schools = {s for s in remaining_schools if remaining_cap[s] > 0}
        return matching
    # Helpers for simulation
    # ------------------------
    def generate_random_market(students, schools):
        prefs = {}
        for i in students:
            ranking = schools[:]
            random.shuffle(ranking)
            prefs[i] = ranking
        priority = {}
        for s in schools:
            ordering = students[:]
            random.shuffle(ordering)
            priority[s] = ordering
```

```
    return prefs, priority
def avg_rank_of_matching(matching, prefs):
    ranks = []
    for i, assigned_school in matching.items():
        r = prefs[i].index(assigned_school) + 1
        ranks.append(r)
    return sum(ranks) / len(ranks)
# Part 3: Simulation
# ------------------------
def run_simulation(N=1000, seed=123):
    random.seed(seed)
    students = [f"i{k}" for k in range(1, 19)]
    schools = [f"s{k}" for k in range(1, 4)]
    capacity = {s: 6 for s in schools}
    da_avgs = []
    ia_avgs = []
    ttc_avgs = []
    for _ in range(N):
        prefs, priority = generate_random_market(students, schools)
        da = deferred_acceptance(students, schools, capacity, prefs, priority)
        ia = immediate_acceptance(students, schools, capacity, prefs, priority)
        ttc = ttc_with_priorities(students, schools, capacity, prefs, priority)
        da_avgs.append(avg_rank_of_matching(da, prefs))
        ia_avgs.append(avg_rank_of_matching(ia, prefs))
        ttc_avgs.append(avg_rank_of_matching(ttc, prefs))
    results = pd.DataFrame({
        "Mechanism": ["DA", "IA", "TTC"],
        "Average student rank (lower is better)": [
            sum(da_avgs)/N,
            sum(ia_avgs)/N,
            sum(ttc_avgs)/N
        ]
    })
    return results
if __name__ == "__main__":
    results = run_simulation(N=1000, seed=123)
    print(results.to_string(index=False))
```